GENERALITAT VALENCIANA
Conselleria d'Educació, Cultura i Esport

IES JUAN DE GARAY
C/ Juan de Garay, 25 46017 Valencia
Tel: 961 20 60 45 Fax: 961 20 60 46
46012963@gva.es www.juandegaray.es

GOBIERNO DE ESPAÑA
MINISTERIO DE EDUCACIÓN Y FORMACIÓN PROFESIONAL

Unión Europea
Fondo Social Europeo
El FSE invierte en tu futuro

UD 06.2B. JAVASCRIPT 2

LM

Ciclos Formativos Grado Superior

Pascual Ligero
pa.ligero2@edu.gva.es

2025/2026

Versión:260227.1923

Licencia

Reconocimiento – NoComercial – CompartirIgual (by-nc-sa): No se permite un uso comercial de la obra original ni de las posibles obras derivadas, la distribución de las cuales se debe hacer con una licencia igual a la que regula la obra original.

Nomenclatura

A lo largo de este tema se utilizarán distintos símbolos para distinguir elementos importantes dentro del contenido. Estos símbolos son:

⚠ Importante

⚛ Interesante

ÍNDICE DE CONTENIDO

1. Selección de Elementos (El Gancho)....5
1.1 Los métodos de selección....5
1.2 Ejemplos....5
1.3 Errores Típicos....6
1.4 Mini-reto (5 min)....6
1.5 Práctica....6
1.5.1 HTML....6
1.5.2 Javascript....7
1.5.2.1 Ver texto en pantalla....7
1.5.2.2 Ver tareas....7
2. Manipulación (Constructor de Legos)....9
2.1 Crear y Borrar....9
2.2 Ejemplos: Lista de Tareas....9
2.3 Errores Típicos....10
2.4 Mini-reto (5 min)....10
2.5 Práctica....10
2.5.1 HTML....11
2.5.2 Javascript....11
3. Eventos (La Magia de la Interacción)....11
3.1 addEventListener y el Objeto event (e)....11
3.2 Delegación de Eventos (Técnica Pro)....13
3.3 Ejemplo 2: Delegación de Eventos (Feed de Red Social)....13
3.4 Errores Típicos....13
3.5 Mini-reto (5 min)....14
3.6 Práctica....14
3.6.1 HTML....14
3.6.2 Javascript....14
3.7 Ampliación del práctica....15
3.7.1 HTML....16
3.7.2 Javascript....16
4. Atributos y Clases (El Estilista)....17
4.1 Herramientas de Estilo....17
4.2 Ejemplos: Modo Oscuro y Likes....17
4.3 Errores Típicos....18
4.4 Práctica....18
4.4.1 HTML....19
4.4.2 Javascript....19
4.5 Mini-reto (5 min)....20
5. Cierre del Módulo 4....21
5.1 Cheat Sheet....21
5.2 Preguntas Rápidas (V/F)....21
5.3 Checklist de Habilidades....22
6. Ejercicios Finales (La Práctica)....22
6.1 Nivel 1: Calentamiento (Fácil)....22
6.2 Nivel 2: Intermedio (Manipulación y Eventos)....23
6.3 Nivel 3: Boss Level (Mini-Apps)....23

UD06.2B. JAVASCRIPT 2

1. SELECCIÓN DE ELEMENTOS (EL GANCHO)

El **DOM** (Document Object Model) es el árbol genealógico de tu web. Para modificar algo (un título, un botón), primero tienes que "agarrarlo" con JavaScript.

1.1 Los métodos de selección

Tenemos tres herramientas principales. Imagina que el HTML es un almacén gigante:

1. **getElementById("id")**: El francotirador. Busca un elemento único por su carnet de identidad (ID). Es el más rápido.
2. **querySelector("selector")**: El explorador. Busca el **primer** elemento que coincida con un selector CSS (clase, id, etiqueta).
3. **querySelectorAll("selector")**: La red de pesca. Trae **todos** los elementos que coincidan en una lista (NodeList).

1.2 Ejemplos

Copia este HTML en tu archivo para probar:

HTML

```
<h1 id="titulo-principal">Hola Mundo</h1>
<button class="btn-action">Click 1</button>
<button class="btn-action">Click 2</button>
<ul id="lista-items">
    <li class="item">Pizza</li>
    <li class="item">Hamburguesa</li>
</ul>
```

JavaScript

```
// JS
// 1. getElementById: Directo y sin el símbolo '#'
const titulo = document.getElementById("titulo-principal");
console.log(titulo.textContent); // "Hola Mundo"

// 2. querySelector: Flexible (necesita '.' para clases o '#' para ids)
// Solo agarra el PRIMER botón que encuentra
const primerBoton = document.querySelector(".btn-action");
```

```
console.log("Primer botón:", primerBoton.textContent);

// 3. querySelectorAll: Agarra TODOS (Lista de Nodos)
const todosLosItems = document.querySelectorAll(".item");

// Recorrer la lista para hacer algo con cada uno
todosLosItems.forEach((item) => {
    console.log("Tengo comida:", item.textContent);
});
```

1.3 Errores Típicos

- Olvidar el # o . en querySelector. *Corrección:* querySelector("boton") busca la etiqueta <boton>, querySelector(".boton") busca class="boton".
- Intentar modificar querySelectorAll directamente:

  todosLosItems.style.color = 'red' // *Corrección:* Es una lista, tienes que recorrerla con un forEach y pintar uno por uno.
- Usar getElementById con #. //*Corrección:* Es getElementById("titulo"), no getElementById("#titulo").

1.4 Mini-reto (5 min)

En el HTML de ejemplo, añade un tercer <li> con clase "item" que diga "Sushi 🍣".

Usa JavaScript para seleccionar **solo** el elemento "Hamburguesa" usando querySelectorAll y el índice del array (pista: es el índice 1). Imprímelo en consola.

1.5 Práctica

El código completo lo tienes en index03.html + tareas03.js

Sigamos con nuestra página de tareas. Por fin vamos a dejar de usar la consola y ya puede informarnos el programa usando el lienzo del navegador.

1.5.1 HTML

Seguimos con el inicial. Todavía no vamos a poner botones ni acciones de otro tipo.

1.5.2 Javascript

1.5.2.1 Ver texto en pantalla

Solo vamos a cambiar la salida para que pueda ver la lista de tareas en la pantalla

Con solo dos líneas puedo escribir en pantalla:

```
//obtengo la posición del HTML
let divTareas=document.getElementById("Tareas")
//modifico su contenido
divTareas.innerHTML="Lo cambio desde javascript"
```

Si en lugar del texto entre comillas le paso las tareas al div no se verá el contenido por que no es capaz de obtener los atributos de los objetos que forman las tareas. Si pruebas la siguiente dirección verás que si que funciona:

```
let pruebas=["Corregir flex!", "Corregir javascript 1","Corregir javascript 2"]
divTareas.innerHTML=pruebas
```

Tareas 0.3. Muestro las tareas en pantalla

Corregir flex!,Corregir javascript 1,Corregir javascript 2

1.5.2.2 Ver tareas

Para poder mostrar tareas vamos a hacer los siguientes pasos:

1) Añadir un método que permita obtener solo los datos de una tarea y devolverlo como una cadena.

2) Con un bucle ir a todas las tareas y juntar la información ellas (punto anterior) en una cadena con toda la información.

3) Escribirlo en el div elegido.

1: Modificar el objeto de tareas que tenemos ,y agregar un nuevo método que devuelva una cadena con toda la información que queremos mostrar en pantalla sobre la tarea deseada. El método lo he llamado mostrarEnHtml()

```
/*
 añade una nueva tarea
```

```
 */
function crearTarea(id, descripcion, fechaLimite, prioridad) {
  const tarea = {
    id,
    descripcion,
    completada: false,
    fechaLimite,
    prioridad
  };

  // usando funciones anonimas flecha es mas sencillo
  // cambiar entre completada o no
  tarea.toggleCompletada = () => {
    tarea.completada = !tarea.completada;
    return tarea;
  };

  // Mostrar en consola
  tarea.mostrarEnConsola = () => {
    const estado = tarea.completada ? '[X]' : '[ ]';
    console.log(`- (${tarea.id}) ${estado} :  ${tarea.descripcion} : $
{tarea.fechaLimite}  (${tarea.prioridad}) `);
  };

  // Devuelve html para Mostrar en pantalla
  tarea.mostrarEnHTML = () => {
    const estado = tarea.completada ? '[X]' : '[ ]';
    let tareaHTML=`- (${tarea.id}) ${estado} :  ${tarea.descripcion} : $
{tarea.fechaLimite}  (${tarea.prioridad}) `
    return tareaHTML
  };

  return tarea;
}
```

2: Con esto podemos recorrer en bucle las tareas creadas, invocar al método que devuelve la cadena

```
/*
    devuelve una cadena con todas las tareas
*/
function mostrarTareasHTML(){
    let todasLasTareas=""
    console.log("Tareas pendientes HTML: ")
    console.log("===================")
    tareas.forEach(element => {
        todasLasTareas=todasLasTareas+element.mostrarEnHTML()+"<br/>"
    });
    return todasLasTareas
}
```

3: y finalmente mostrarlo en pantalla.

```
divTareas.innerHTML=mostrarTareasHTML()
```

Tareas 0.3. Muestro las tareas en pantalla

- (1) [ ] : Estudiar arrays: forEach/map : 2026-02-23 (alta)
- (2) [ ] : 5 ejercicios con objetos JS : 2026-02-24 (alta)
- (3) [ ] : DOM: Manipular elementos : 2026-02-25 (media)
- (4) [ ] : DOM: Eventos : 2026-02-26 (baja)

2. MANIPULACIÓN (CONSTRUCTOR DE LEGOS)

Pintar de la forma anterior en el árbol HTML es sucio y difícil de mantener. Javascript no solo nos dejará leer el HTML, también podemos **crearlo** desde cero y **borrarlo**.

2.1 Crear y Borrar

1. **document.createElement("etiqueta")**: Crea un "fantasma" en la memoria (no se ve aún).

   ```
   const nuevaTarea = document.createElement("li");
   ```

2. **padre.append(hijo)**: Lo pega al final del contenedor. (También existe prepend para ponerlo al principio).

   ```
   const lista = document.getElementById("lista");
   ….. .
   lista.append(nuevaTarea)
   ```

3. **elemento.remove()**: Lo elimina del universo.

   ```
   lista.remove()
   ```

2.2 Ejemplos: Lista de Tareas

HTML

```
<div id="contenedor-tareas">
    <h3>Mis Tareas</h3>
    <ul id="lista"></ul>
</div>
```

JavaScript

```
// JS
```

```
const lista = document.getElementById("lista");

// 1. Crear el elemento (está en el limbo)
const nuevaTarea = document.createElement("li");

// 2. Rellenarlo
nuevaTarea.textContent = "Aprender DOM";
nuevaTarea.classList.add("tarea-urgente"); // Le damos estilo (si hubiera CSS)

// 3. Inyectarlo en la web
// append: Lo pone al final
lista.append(nuevaTarea);

// prepend: Lo pone al principio
const tareaTop = document.createElement("li");
tareaTop.textContent = "Comprar café";
lista.prepend(tareaTop);

// 4. Eliminar elementos
// Vamos a borrar la tarea de "Aprender DOM" después de 3 segundos
setTimeout(() => {
    nuevaTarea.remove();
    console.log("Tarea eliminada");
}, 3000);
```

2.3 Errores Típicos

- Confundir append con appendChild. *Corrección:* Usa append. Es más moderno, permite agregar texto directo (lista.append("Hola")) y varios elementos a la vez. appendChild es más viejo y estricto.
- Crear un elemento y no agregarlo. *Corrección:* Si no haces append o prepend, el elemento existe en JS pero nunca se pinta en el HTML.

2.4 Mini-reto (5 min)

Crea un div con createElement. Ponle dentro el texto "Soy una alerta". Insértalo en el document.body al **principio** de todo (prepend).

2.5 Práctica

El código completo lo tienes en index04.html + tareas04.js

Vamos a mejorar nuestro gestor de tareas, crearemos un árbol de tareas del estilo

```
<ul>
        <li>tarea 1 </li>
        <li>…. </li>
        <li>tarea n </li>
</ul
```

2.5.1 HTML

Sigue igual, vamos a generar el código en tiempo real desde javascript

2.5.2 Javascript

Modificaremos la función mostrarTareasHTML() para que en lugar de insertar texto genere un árbol html de ul con cada tarea en un li

```
/*
    devuelve una LISTA con todas las tareas
*/
function mostrarTareasHTML(){
    console.log("Tareas pendientes HTML: ")
    console.log("===================")
    let miLista= document.createElement("ul");
    tareas.forEach(element => {
        let miItem= document.createElement("li");
        miItem.innerHTML=element.mostrarEnHTML()
        miLista.appendChild(miItem)
    });
    console.log(miLista)
    return miLista
}
```

Ahora ya no nos vale usar innerHTML por que no vamos a escribir texto en el div de la pantalla, tenemos que añadir el ul como hijo de ese div!:

```
divTareas.append(mostrarTareasHTML())
```

3. EVENTOS (LA MAGIA DE LA INTERACCIÓN)

Los eventos son el sistema nervioso. Detectan clics, teclas, scroll, envíos...

3.1 addEventListener y el Objeto event (e)

Puedes hacer que cualquier elemento HTML del documento “interaccione” con el

javascript asociado, de forma que al hacer un “tipo” de acción se llame a una función escrita.

Para ello, en el javascript, al elemento que queramos, por ejemplo un botón, usando el getElementById(“botonAdd”) lo añadiremos a una variable y después le asignaremos una escucha de eventos. Te lo pongo en código después de la definición.

- **elemento.addEventListener("tipo", funcion)**: "Cuando pase X, ejecuta Y".

El código completo lo tienes en eventos06.html + eventos.js

```
html:
<body>
    <h1>Probando eventos</h1>
    <form id="form-comentario" action="">
        <input type="text" id="input-texto" placeholder="Escribe un
comentario...">
        <button type="submit">Enviar</button>
    </form>
    <ul id="lista-comentarios"></ul>

    <script src="js/eventos.js"></script>
</body>

javascript:
// apunto a los elementosdel HTML que necesito
const formulario = document.getElementById("form-comentario");
const input = document.getElementById("input-texto");
const listaComentarios = document.getElementById("lista-comentarios");
const boton=document.getElementById("botonAdd")

// registro el boton para añadir comentarios
boton.addEventListener("click", (e) => {
    // Evito que el navegador recargue la pagina debido al submit
    e.preventDefault();
    // Si no pones la línea de arriba, la página parpadea y se borra todo.
    // Obtener lo que escribió el usuario
    const texto = input.value;

    // Validación básica, no hago nada si está vacio
    if (texto === "") return;

    // Crear nuevo li y añadir al DOM
    const nuevoLi = document.createElement("li");
    nuevoLi.textContent = texto;
    listaComentarios.append(nuevoLi);

    // Limpiar el input
    input.value = "";
    console.log("Comentario enviado sin recargar ");

})
```

- **e.preventDefault()**: **OBLIGATORIO** aprenderlo. Evita el comportamiento por

defecto del navegador.

- *Ejemplo clásico:* Al enviar un <form>, el navegador recarga la página. e.preventDefault() lo frena para que tú manejes los datos con JS sin recargar.

3.2 Delegación de Eventos (Técnica Pro)

Si tienes una lista con 100 ítems, **no pongas 100 listeners**. Pon **uno solo** en el contenedor padre y pregunta "¿a quién le diste click?" usando e.target.

3.3 Ejemplo 2: Delegación de Eventos (Feed de Red Social)

El código completo lo tienes en eventos07.html + eventos2.js

HTML

```
<body>
    <h1>Probando eventos 2</h1>
    <ul id="feed">
        <li>Meme de gatos <button class="btn-borrar">Borrar</button></li>
        <li>Noticia Tech <button class="btn-borrar">Borrar</button></li>
    </ul>

    <script src="js/eventos2.js"></script>
</body>
```

JavaScript

```
const feed = document.getElementById("feed");

// Un solo portero para todo el edificio
feed.addEventListener("click", (e) => {
    // e.target es EXACTAMENTE lo que clickaste (puede ser el texto, el borde, el
botón)

    // ¿Clickaste en algo que tenga la clase 'btn-borrar'?
    if (e.target.classList.contains("btn-borrar")) {
        // Buscamos al padre (el <li> completo) y lo matamos
        const tweet = e.target.parentElement; // o e.target.closest('li') que es
más seguro
        tweet.remove();
        console.log("Tweet eliminado mediante delegación 🗑️");
    }
});
```

3.4 Errores Típicos

- Ejecutar la función al ponerla: btn.addEventListener("click", saludar()). *Corrección:* Quita los paréntesis saludar o usa () => saludar().

- Olvidar input.value. *Corrección:* input.textContent no funciona en inputs de escribir, es .value.

3.5 Mini-reto (5 min)

Crea un input de texto. Añade un evento input (se dispara cada vez que escribes una letra) que muestre en un span cuántos caracteres llevas escritos en tiempo real.

3.6 Práctica

El código completo lo tienes en index08.html + tareas05.js

3.6.1 HTML

En esta parte de la práctica vamos a incluir un formulario para añadir nuevas tareas

```
<h1>Tareas 0.5. Muestro/Añado/Completo/Borro las tareas con estilos</h1>
<div id="Tareas">aquí irán las tareas en la versión 0.5</div>
<form id="formTarea">
  <fieldset>
    <legend>Nueva tarea</legend>

    <label for="tareaDescripcion">Descripción</label>
    <input id="tareaDescripcion" name="descripcion" type="text" minlength="1"
maxlength="200" required/>

    <label for="tareaFechaLimite">Fecha límite</label>
    <input id="tareaFechaLimite" name="fechaLimite" type="date" required />

    <label for="tareaPrioridad">Prioridad</label>
    <select id="tareaPrioridad" name="prioridad" required>
        <option value="">Elige una…</option>
        <option value="baja">Baja</option>
        <option value="media">Media</option>
        <option value="alta">Alta</option>
    </select>

    <button type="submit">Crear tarea</button>
    <button type="reset">Limpiar</button>
  </fieldset>
</form>
<script src="js/tareas05.js"></script>
```

3.6.2 Javascript

Además de atender los eventos tenemos que cambiar y añadir algunas funciones.

1) Necesito una función que me diga el id de la próxima tarea.

```
/*
    Devuelve el id para nueva tarea buscando el ultimo creado
*/
function devuelveNuevoId(){
```

```
    let id=0
    tareas.forEach(element => {
        if(element.id>id){
            id=element.id
        }
    });
    return id+1
}
```

2) La función crearTarea será la misma que antes, pero ahora no le paso el id, sino que lo pedirá a la función anterior:

```
function crearTarea(descripcion, fechaLimite, prioridad) {
  //obtengo el próximo id
  let id=devuelveNuevoId()
```

3) Atiendo los eventos para añadir tareas:

```
// eventos
  const form = document.querySelector("#formTarea");

//atiende a la creación de una nueva tarea
form.addEventListener("submit", (e) => {
    e.preventDefault(); // evita recargar / enviar el form


    const descripcion = form.elements.descripcion.value.trim();
    const fechaLimite = form.elements.fechaLimite.value; // "YYYY-MM-DD"
    const prioridad = form.elements.prioridad.value;
    const tarea = crearTarea(descripcion, fechaLimite, prioridad);

    tareas.push(tarea)
    // borrar las anteriores
    divTareas.innerHTML = "";
    // refrescar
    divTareas.append(mostrarTareasHTML())

    form.reset();
});
```

4) Para que no aparezca la lista de tareas vacías:

```
//creo las tareas de prueba
tareas.push(crearTarea("Estudiar arrays: forEach/map", "2026-02-23","alta"));
tareas.push(crearTarea("5 ejercicios con objetos JS", "2026-02-24", "alta"));
tareas.push(crearTarea("DOM: Manipular elementos", "2026-02-25", "media"));
tareas.push(crearTarea("DOM: Eventos", "2026-02-26", "baja"));

// borrar el div si contiene algo para poder añadir
divTareas.innerHTML = "";

//pinto las tareas de ejemplo
divTareas.append(mostrarTareasHTML())
```

3.7 Ampliación del práctica

El código completo lo tienes en index09.html + tareas06.js

Nos falta añadir una forma de marcar las tareas como hechas y de eliminarlas tareas

3.7.1 HTML

Añadimos a cada tarea un botón para marcar como hecha y otro para borrar

Como no sabemos cuantas tareas tenemos y además se generan nuevas cada vez, los botones tendremos que crearlos desde javascript

3.7.2 Javascript

Vamos a modificar la función dentro del objeto tarea para que añada los dos botones cada vez que queramos mostrarlo. En cada botón añadimos un atributo data-id con el id de la tarea que queremos borrar/modificar

```
// Devuelve html para Mostrar en pantalla
tarea.mostrarEnHTML = () => {
  const estado = tarea.completada ? '[X]' : '[ ]';
  let tareaHTML=`- (${tarea.id}) ${estado} :  ${tarea.descripcion} : $
{tarea.fechaLimite}  (${tarea.prioridad}) <input type="button" data-id="$
{tarea.id}" class="btn-borrar" data-action="borrar" value="Borrar"> <input
type="button" data-id="${tarea.id}" class="btn-toggle" data-action="mark"
value="Cambia Estado">`
  return tareaHTML
};
```

Puedes comprobar que hemos usado dos atributos que hasta ahora no habíamos usado **data-id** y **data-action**, son *atributos de datos personalizados* de HTML (se escriben como `data-*`) que te permiten "pegar" información extra a un elemento del DOM sin inventarte atributos raros ni abusar de `id` o `class`.

Puedes usar **tantos atributos `data-*` como quieras** en un mismo elemento, siempre que sigan el patrón `data-<nombre>="valor"`.

El `<nombre>` lo defines tú (por ejemplo `data-prioridad`, `data-fecha-limite`, `data-user-id`, etc.) y el valor es texto (cadena) en HTML.

Desde JavaScript puedes leerlos fácilmente con la propiedad `elemento.dataset` (por ejemplo elemento`.dataset.id` o elemento`.dataset.action`)

`dataset` te devuelve cadenas (strings), así que si esperas un número normalmente harás `Number(elemento.dataset.id) para convertirlo a número`

Ahora voy a recoger los eventos y realizar algo en función del action sobre el id:

```
// escucho los clicks en el div  Tareas
divTareas.addEventListener('click', (e) => {
```

```
    // guardo el elemnto clickado
    let element = e.target;

    // obtiene la tarea y que quiero hacer con ella
    const id = Number(element.dataset.id);
    const action = element.dataset.action;

    if (action === 'borrar') {
        borrarTareaPorId(id);
    } else if (action === 'mark') {
        toggleTareaPorId(id);
    }
});
```

Tareas 0.5. Muestro/Añado/Completo/Borro las tareas con estilos

- - (1) [ ] : Estudiar arrays: forEach/map : 2026-02-23 (alta) Borrar Cambia Estado
- - (2) [ ] : 5 ejercicios con objetos JS : 2026-02-24 (alta) Borrar Cambia Estado
- - (3) [ ] : DOM: Manipular elementos : 2026-02-25 (media) Borrar Cambia Estado
- - (4) [ ] : DOM: Eventos : 2026-02-26 (baja) Borrar Cambia Estado

Nueva tarea
Descripción Fecha límite dd / mm / aaaa Prioridad Elige una... Crear tarea Limpiar

4. ATRIBUTOS Y CLASES (EL ESTILISTA)

Nunca (o casi nunca) uses JS para escribir CSS directo. Es sucio. Lo profesional es tener clases en CSS y usar JS para **activarlas o desactivarlas**.

4.1 Herramientas de Estilo

1. **classList**: .add("clase"), .remove("clase"), .toggle("clase") (interruptor: si está la quita, si no la pone), .contains("clase") (¿la tiene?).
2. **style.propiedad**: Para cambios puntuales (ej: coordenadas dinámicas). Se escribe en **camelCase** (backgroundColor, no background-color).
3. **dataset**: Para guardar datos ocultos útiles para la lógica (data-id, data-precio, data-user), lo acabamos de ver.

4.2 Ejemplos: Modo Oscuro y Likes

El código completo lo tienes en estilos10.html + tareas07.js

HTML

```
<style>
    .dark-mode { background-color: #333; color: white; }
    .corazon-rojo { color: red; font-weight: bold; }
```

```
</style>

<button id="btn-tema">Cambiar Tema 🌙</button>
<div id="post-1" data-id="123" data-autor="milena">
    <p>Post increíble sobre JS</p>
    <button id="btn-like">Like ❤️</button>
</div>
```

JavaScript

```
// JS
const btnTema = document.getElementById("btn-tema");
const btnLike = document.getElementById("btn-like");
const post = document.getElementById("post-1");

// 1. classList.toggle (El interruptor, quita o pone según que clase tenía antes)
btnTema.addEventListener("click", () => {
    document.body.classList.toggle("dark-mode");
});

// 2. classList.add/remove
btnLike.addEventListener("click", () => {
    const yaTeniaLike = btnLike.classList.contains("corazon-rojo");

    if (yaTeniaLike) {
        btnLike.classList.remove("corazon-rojo");
        console.log("Like quitado 💔");
    } else {
        btnLike.classList.add("corazon-rojo");

        // 3. dataset: Leer datos ocultos del padre
        // dataset.id lee data-id, dataset.autor lee data-autor
        console.log(`Diste like al post ${post.dataset.id} de ${post.dataset.autor}`);
    }
});
```

4.3 Errores Típicos

- Escribir elemento.class = "rojo". *Corrección:* Eso borra todas las clases anteriores. Usa classList.add("rojo").
- Usar style.background-color. *Corrección:* En JS el guion es resta. Usa camelCase: style.backgroundColor.

4.4 Práctica

El código completo lo tienes en estilos11.html + tareas08.js

Vamos a dejarlo bonito, vamos a poner cada tarea con un checkbox y con estilos

4.4.1 HTML

Añadimos una hoja de estilos

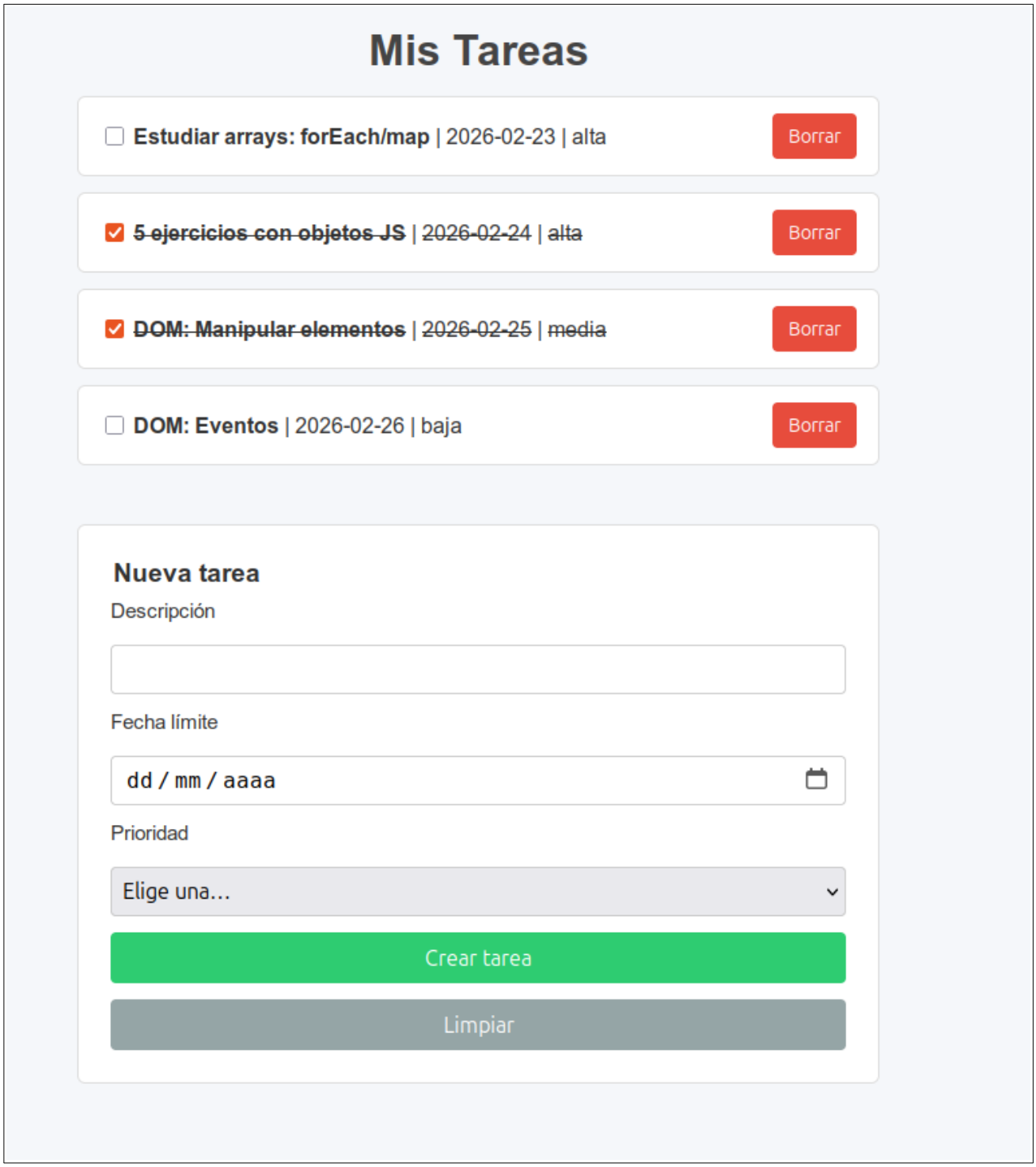
```
<link rel="stylesheet" href="css/style.css">
```

4.4.2 Javascript

Mejoramos el aspecto de las tareas:

```
// Devuelve html para Mostrar en pantalla
    tarea.mostrarEnHTML = () => {
        const estado = tarea.completada ? 'checked' : '';
      // añado un estilo tachado si esta completada
        const estiloCompelatada = tarea.completada ? 'completed' : '';

        // wrap the main info in a span so flexbox can manage spacing
        let tareaHTML = `<span class="info"><input type="checkbox" class="chk-
completada" data-id="${tarea.id}" data-action="mark" ${estado}> <span
class="descripcion ${estiloCompelatada}">${tarea.descripcion}</span> | <span
class="fecha ${estiloCompelatada}">${tarea.fechaLimite}</span>  | <span
class="prioridad ${estiloCompelatada}">${tarea.prioridad}</span></span>` +
        `<input type="button" data-id="${tarea.id}" class="btn-borrar" data-
action="borrar" value="Borrar">`;
        return tareaHTML;
    };
```



4.5 Mini-reto (5 min)

Tienes un .

Haz que al pulsar un botón, el ancho de la caja aumente 10px cada vez usando style.width.

Pista: Necesitas una variable let ancho = 100; y sumarle.

5. CIERRE DEL MÓDULO 4

5.1 Cheat Sheet

- **Seleccionar ID:** document.getElementById("nombre").
- **Seleccionar Clase:** document.querySelector(".nombre").
- **Crear:** document.createElement("div").
- **Añadir:** padre.append(hijo).
- **Clases:** elemento.classList.toggle("activa").
- **Input valor:** input.value.
- **Texto normal:** elemento.textContent.
- **Evitar recarga:** e.preventDefault() en el evento submit.
- **Delegación:** Listener en el padre + if (e.target.matches(".clase")).

5.2 Preguntas Rápidas (V/F)

1. querySelector devuelve una lista con todos los elementos que coincidan. (V/F)
2. document.getElementById necesita que le pases el símbolo #. (V/F)
3. e.preventDefault() se usa para evitar que un formulario recargue la página. (V/F)
4. innerHTML es más seguro que textContent. (V/F)
5. append permite añadir texto directamente sin crear un nodo de texto. (V/F)
6. Para borrar un elemento usamos delete(). (V/F)
7. classList.toggle añade una clase si no existe, y la quita si existe. (V/F)
8. Los atributos data-id se leen en JS con dataset.id. (V/F)
9. Es mejor poner 100 listeners en 100 botones que 1 listener en el padre. (V/F)
10. El evento input se dispara cuando el usuario termina de escribir y sale del campo. (V/F)

5.3 Checklist de Habilidades

- [ ] Sé seleccionar un elemento por ID y por clase.
- [ ] Sé crear un elemento nuevo y meterlo dentro de otro.
- [ ] Sé cambiar el texto y las clases CSS de un elemento.
- [ ] Sé hacer que un botón ejecute código al hacer click.
- [ ] Sé capturar lo que el usuario escribe en un input.
- [ ] Entiendo por qué debo usar e.preventDefault() en formularios.

---

6. EJERCICIOS FINALES (LA PRÁCTICA)

6.1 Nivel 1: Calentamiento (Fácil)

1. **El Cambia-Texto:**
   - *HTML:* <h1 id="titulo">Hola</h1> <button id="btn">Cambiar</button>
   - *JS:* Al clickar, cambia el texto del H1 a "¡Adiós!".
2. **El Interruptor de Color:**
   - *HTML:* Un div cuadrado gris.
   - *JS:* Al pasar el mouse por encima (mouseenter), ponlo rojo. Al salir (mouseleave), ponlo gris otra vez. (Usa style.backgroundColor).
3. **Espejo Mágico:**
   - *HTML:* Un input text y un p vacío.
   - *JS:* Todo lo que escribas en el input debe aparecer idéntico en el párrafo en tiempo real (evento input).
4. **Clase Toggle:**
   - *HTML:* Un texto y un botón. CSS: .oculto { display: none; }.
   - *JS:* Al clickar el botón, oculta o muestra el texto usando classList.toggle.
5. **Lista Estática:**
   - *HTML:* Una lista ul con 3 frutas.
   - *JS:* Selecciona todas las frutas con querySelectorAll, recórrelas con forEach

y añade un emoji "🍎" al final del texto de cada una.

6.2 Nivel 2: Intermedio (Manipulación y Eventos)

6. Contador de Clics:

- *HTML:* Botón que dice "Clicks: 0".
- *JS:* Variable contador = 0. Al clickar, suma 1 y actualiza el texto del botón.

7. Galería de Imágenes (Atributos):

- *HTML:* Una imagen grande img src="img1.jpg" y tres botones debajo: "Ver Img 1", "Ver Img 2", "Ver Img 3".
- *JS:* Al clickar cada botón, cambia el atributo src de la imagen grande.

8. Validador de Password Simple:

- *HTML:* Input password y botón enviar.
- *JS:* Al clickar enviar, si el password tiene menos de 5 caracteres, ponle un borde rojo (style.border). Si está bien, borde verde.
- *Pista:* input.value.length.

9. Lista de Tareas (Añadir):

- *HTML:* Input text, botón "Añadir", ul vacía.
- *JS:* Al clickar añadir, crea un li, ponle el texto del input y agrégalo al ul. **Importante:** Limpia el input después.

10. Eliminar Tarea (Individual):

- *JS:* En el ejercicio anterior, cuando crees el li, añádele dentro un botón que diga "X". Al clickar esa X, el li debe borrarse (remove()).

11. El Dataset Detector:

- *HTML:* 3 botones con data-precio="100", data-precio="200", etc.
- *JS:* Un solo listener (delegación) que, al clickar un botón, haga console.log del precio de ese producto.

6.3 Nivel 3: Boss Level (Mini-Apps)

12. Comentarios tipo Twitter (Top Insert):

- *Enunciado:* Formulario para escribir tweet. Al enviar (submit +

preventDefault), el tweet debe aparecer **arriba del todo** de la lista (prepend).

- *Restricción:* Si el tweet está vacío, no hagas nada. Añade la hora actual al tweet (new Date().toLocaleTimeString()).

13.Carrito de Compras (Solo DOM):

- *HTML:* Lista de productos (Nombre + Precio + Botón "Comprar"). Un span que diga "Total: $0".
- *JS:* Usa delegación de eventos para detectar click en "Comprar".
- *Lógica:* Lee el precio del producto (desde el texto o dataset), súmalo a una variable total global y actualiza el texto del span.

14.Buscador/Filtro de Nombres:

- *HTML:* Un input "Buscar..." y una lista ul con 10 nombres (li) fijos.
- *JS:* Evento keyup o input.
- *Lógica:* Recorre todos los li. Si el texto del nombre **incluye** lo que hay en el input, quítale la clase .hidden. Si no lo incluye, pónsela (.hidden { display: none }).
- *Pista:* li.textContent.toLowerCase().includes(input.value.toLowerCase()).